Case Studies

Case Study 1. FBI Virtual Case File System

After the **September 11 attacks**, the **Federal Bureau of Investigation** realized that their existing case management system was outdated and inefficient. Agents were still using **paper files and manual processes** to manage criminal investigations. To modernize their operations, the FBI launched a project called the **Virtual Case File (VCF) System**.The system was expected to:

- Digitize investigation records
- Allow agents to search and share case information
- Improve collaboration between FBI offices
- Speed up criminal investigations

The project was estimated to cost **about $170 million**.

Problems in the Project

1. Constant Requirement Changes
2. Lack of Clear Planning
3. Poor Communication
4. Technical Issues

After spending **about $170 million**, the project was **abandoned in 2005**.

Case 11. Denver International Airport Baggage System Failure

Automated **baggage handling software system**.

Project cost:

$193 million

But during testing:

- Bags were **thrown off tracks**
- Bags were **lost**
- Bags were **destroyed**

In demonstrations, luggage was **flying everywhere**.

Root Cause

1. Requirements kept **changing during development**.
2. Airport authorities **added features late**.
3. Software and hardware teams were **poorly coordinated**.
4. No **incremental development**.
5. Poor **risk assessment**.

Case Study III.The Disastrous Launch of Healthcare.gov

The website was created to allow citizens to **buy health insurance online**.

Project cost: **$500+ million**

When the website launched:

- Users could **not log in**
- Pages **crashed frequently**
- System **froze under load**

Only **6 users successfully registered on the first day**.

Root Cause

1. **Too many contractors** working independently.
2. Poor **coordination between teams**.
3. No **end-to-end system testing before launch**.
4. Requirements kept **changing frequently**.
5. Unrealistic **deadline pressure**.

PROJECT MANAGEMENT

PROJECT MANAGEMENT

- Project management involves the planning, monitoring, and control of the people, process, and events that occur as software evolves from a preliminary concept to full operational deployment.

Scenario: Healthcare IoT App

What went right:

- Clear requirements
- Good architecture
- Realistic timeline

What went wrong:

- Poor communication
- Dominating team member

Outcome:

- Misunderstanding
- Rework

Scenario: Mobile Banking App

What went right:

- Good team
- Agile process

What went wrong:

- Wrong features built

Outcome:

- Low adoption

Scenario: E-commerce Platform

What went right:

- Skilled team
- Clear requirements

What went wrong:

- No version control discipline
- No testing strategy

Outcome:

- Bugs
- Conflicts

Scenario: Smart City Traffic System

What went right:

- Strong team
- Good requirements

What went wrong:

- Poor planning
- Unrealistic deadlines

Outcome:

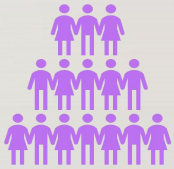
- Delays
- Budget overrun

THE MANAGEMENT SPECTRUM

- 4 P's
  - People
  - Product
  - Process
  - Project

PEOPLE

People Capability Maturity Model
(People-CMM)



The Stakeholders

Senior managers
Project (technical) managers
Practitioners
Customers
End users

- Team Leader
- The Software Team depends on following factors:
  - Difficulty of the problem to be solved
  - Size
  - Team lifetime
  - Degree to which the problem can be modularized
  - Quality and reliability
  - Rigidity of the delivery date
  - Communication required for the project

ORGANIZATIONAL PARADIGMS

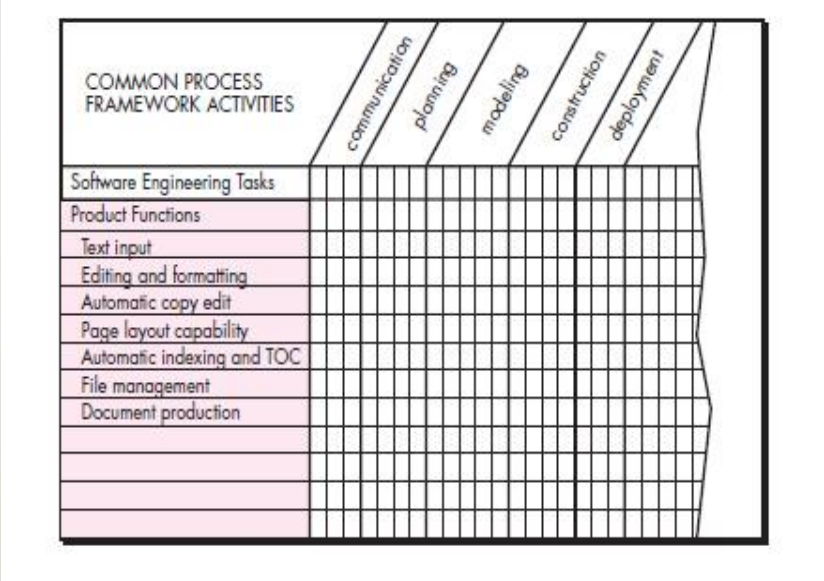
- Closed paradigm
- Random paradigm
- Open paradigm
- Synchronous paradigm

PRODUCT

- Product objectives and scope should be established
- Software Scope:
  - **Context.**
  - **Information objectives**
  - **Function and performance**
- Problem Decomposition

PROCESS

- A number of different task sets—tasks, milestones, work products, and quality assurance points



PROJECT

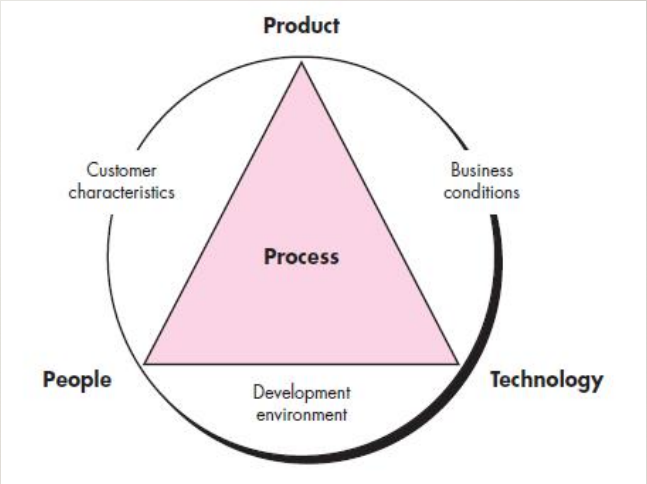
- Approach for planning, monitoring, and controlling the project.
- Five-part approach to software projects:
  - Start on the right foot
  - Maintain momentum.
  - Track progress
  - Make smart decisions
  - Conduct a postmortem analysis

THE W5HH PRINCIPLE

- Why is the system being developed?
- What will be done?
- When will it be done?
- Who is responsible for a function?
- Where are they located organizationally?
- How will the job be done technically and managerially?
- How much of each resource is needed?.

PROCESS METRICS

- *Process metrics* are collected across all projects and over long periods of time.
- lead to long-term software process improvement.
- **Private Metrics**
- **Public metrics**



PROJECT METRICS

- Metrics collected from past projects are used as a basis from which effort and time estimates are made for current software work
- To minimize the development schedule by making the adjustments necessary to avoid delays and, mitigate potential problems and risks.
- project metrics are used to assess product quality on an ongoing basis and, when necessary, modify the technical approach to improve quality.

MEASUREMENT

- Direct Measure
  - Cost
  - Effort
- Indirect Measure
  - Functionality,
  - Quality
  - Complexity,
  - Efficiency, Reliability, Maintainability etc.

SOFTWARE MEASUREMENT

- process, project, and product metrics
- Product metrics that are private to an individual are often combined to develop project metrics that are public to a software team.
- Project metrics are then consolidated to create process metrics that are public to the software organization as a whole.

PRODUCT METRICS

- METRICS FOR THE REQUIREMENTS MODEL
  - Function based metrics
  - Metrics for Specification Quality
- METRICS FOR THE DESIGN MODEL
  - Architectural Design Metrics
- METRICS FOR SOURCE CODE-Halstead's Theory
- METRICS FOR TESTING
- METRICS FOR MAINTENANCE

METRICS FOR THE REQUIREMENTS MODEL

FUNCTION BASED METRICS

- Predicting the “size” of the resultant system
- A means for measuring the functionality delivered by a system.
- Information domain values are defined as:
  - **Number of external inputs (Eis)-** originates from a user or is transmitted from another application and provides distinct application-oriented data or control information.
  - **Number of external outputs (EOs).** Each *external output* is derived data within the application that provides information to the user. In this context external output refers to reports, screens, error messages, etc. Individual data items within a report are not counted separately.
  - **Number of external inquiries (EQs).** An *external inquiry* is defined as an online input that results in the generation of some immediate software response in the form of an online output (often retrieved from an ILF)
  - **Number of internal logical files (ILFs).** Each *internal logical file* is a logical grouping of data that resides within the application’s boundary and is maintained via external inputs.
  - **Number of external interface files (EIFs).** Each *external interface file* is a logical grouping of data that resides external to the application but provides information that may be of use to the application.

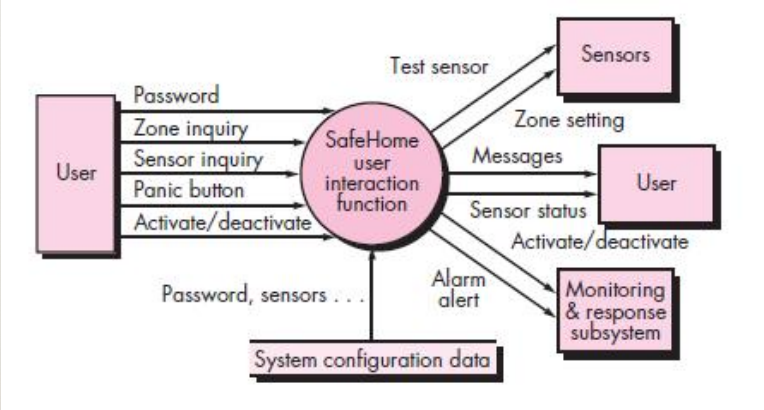
- To compute function points (FP), the following relationship is used:

FP = count total * $[0.65 + 0.01 * \sum F_i]$

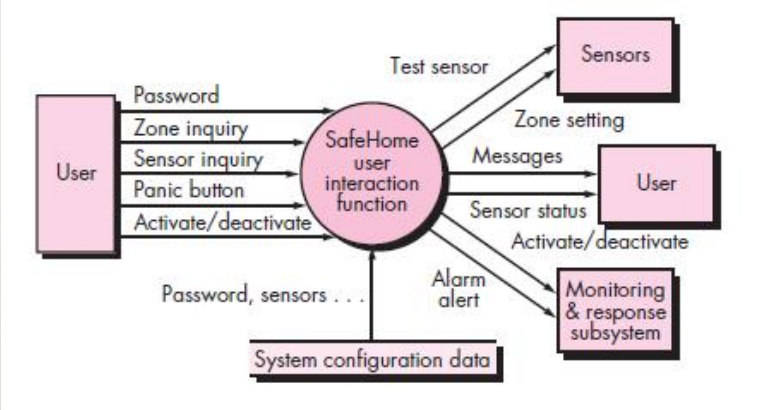
- The *Fi* (*i* 1 to 14) are *value adjustment factors* (VAF) based on responses to the following such questions:
  - Are the ILFs updated online?
  - Are the inputs, outputs, files, or inquiries complex?
  - Is the internal processing complex?
  - Is the code designed to be reusable?
- On a scale of ranges from 0 (not important or applicable) to 5 (absolutely essential

Information Domain Value	Count		Weighting factor Simple	Average	Complex		
External Inputs (EIs)		×	3	4	6	=	
External Outputs (EOs)		×	4	5	7	=	
External Inquiries (EQs)		×	3	4	6	=	
Internal Logical Files (ILFs)		×	7	10	15	=	
External Interface Files (EIFs)		×	5	7	10	=	
Count total							

The function manages user interaction, accepting a user password to activate or deactivate the system, and allows inquiries on the status of security zones and various security sensors. The function displays a series of prompting messages and sends appropriate control signals to the users as well as for differerent component of the system such as to test sensors, to change the zone settings, to activate and deactivate the monitoring systems and to change alarm settings.



Information Domain Value	Count		Weighting factor Simple	Average	Complex		
External Inputs (EIs)	3	×	3	4	6	=	9
External Outputs (EOs)	2	×	4	5	7	=	8
External Inquiries (EQs)	2	×	3	4	6	=	6
Internal Logical Files (ILFs)	1	×	7	10	15	=	7
External Interface Files (EIFs)	4	×	5	7	10	=	20
Count total							50



FP= 50 * [0.65 + (0.01 * 46)]= 56

	average
User Input = 50	4
User Output = 40	5
User Inquiries = 35	4
User Files = 6	10
External Interface = 4	7

F=42

- Function point=????

	average
User Input = 50	4
User Output = 40	5
User Inquiries = 35	4
User Files = 6	10
External Interface = 4	7

F=42

- Function point= 671.96

A software company is developing an Online Food Delivery System. During requirement analysis, the following components were identified: 5 External Inputs (EI) for placing orders, updating menu items, and registering users; 4 External Outputs (EO) such as order confirmation and sales reports; 3 External Inquiries (EQ) for checking order status and searching restaurants; 2 Internal Logical Files (ILF) including customer and order databases; and 1 External Interface File (EIF) connected to a payment gateway system. Using standard average complexity weights (EI=4, EO=5, EQ=4, ILF=10, EIF=7) and the system's Value Adjustment Factor (VAF) based on general system characteristics is 40. Calculate the final Function Points (FP) using the formula FP = Count × (0.65 + 0.01 × VAF).

METRICS FOR SPECIFICATION QUALITY

$$n_r = n_f + n_{nf}$$

- *Specificity* $Q_1 = n_{UI} / n_n$
- *Completeness* $Q2 = n_u / (n_i * n_s)$

METRICS FOR THE DESIGN MODEL

- ***Structural complexity*** of a module *i* is defined in the following manner:

$$S_i = f_{out}^2(\mathrm{i})$$

- ***Data complexity*** provides an indication of the complexity in the internal interface for a module *i* and is defined as:

$$D_i = v(i)/(f_{out}(\mathrm{i})+1)$$

- ***System complexity*** is defined as the sum of structural and data complexity, specified as

$$C(i) = S(i) + D(i)$$

METRICS FOR SOURCE CODE- HALSTEAD'S THEORY

$n1$ = number of distinct operators that appear in a program
$n2$ = number of distinct operands that appear in a program
$N1$ =total number of operator occurrences
$N2$ = total number of operand occurrences

- **Length**

$$N = n_1 \log_2 n_1 + n_2 \log_2 n_2$$

- **Volume**

$$V = N \log_2 (n_1 + n_2)$$

- **Level**

$$L = \frac{2}{n_1} * \frac{n_2}{N_2}$$

- **Difficulty**

$$D = 1/L$$

- **Effort**

$$E = V*D$$

```
voidsort( int*a, intn )
 { int i, j, t;
            if( n <2 ) return;
            for( i=0 ; i <n-1; i++)
            { for( j=i+1 ; j <n ; j++)
                        {if( a[i] > a[j])
                                    { t=a[i];
                                      a[i] =a[j];
                                      a[j]=t;
                        } } }
```

```
voidsort( int*a, int n )
 { int i, j, t;
        if( n <2 ) return;
        for( i=0 ; i <n-1; i++)
        { for( j=i+1 ; j <n ; j++)
                {if( a[i] > a[j])
                        { t=a[i];
                          a[i] =a[j];
                          a[j]=t;
                } } }
```

Operators N1 = 50 n1 = 17
Operands N2 = 30 n2 = 8

```
int sort (int x[ ], int n)

{
int i, j, save, im1;
/*This function sorts array x in ascending order */
If (n< 2) return 1;
for (i=2; i< =n; i++)
{
im1=i-1;
for (j=1; j< =im1; j++)
if (x[i] < x[j])
{
Save = x[i];
x[i] = x[j];
x[j] = save;
}
}
return 0;
}
```

V = 417.23 bits

Operators	Occurrences	Operands	Occurrences
int	4	sort	1
()	5	x	7
,	4	n	3
[]	7	i	8
if	2	j	7
<	2	save	3
;	11	im1	3
for	2	2	2
=	6	1	3
–	1	0	1
<=	2	–	–
++	2	–	–
return	2	–	–
{}	3	–	–
n1=14	N1=53	n2=10	N2=38

METRICS FOR TESTING

- Halstead Metrics Applied to Testing

$$PL = \frac{1}{\left({}^{n_1}/_{2}\right) * \left({}^{N_2}/_{n_2}\right)}$$

$$e = \frac{V}{PL}$$

METRICS FOR MAINTENANCE

MT = number of modules in the current release

Fc = number of modules in the current release that have been changed

Fa = number of modules in the current release that have been added

Fd = number of modules from the preceding release that were deleted in the current release

- *software maturity index* (SMI) that provides an indication of the stability of a software product

$$\text{SMI} = \frac{M_T - (F_c + F_a + F_d)}{M_T}$$